jupyter Untitled8 Last Checkpoint: 37 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

•[6]:
```python
#Two Sum (Optimized)
def twoSum(nums, target):
    hash_map = {}  # Key: number, Value: index

    for i, num in enumerate(nums):
        complement = target - num
        if complement in hash_map:
            return [hash_map[complement], i]
        hash_map[num] = i  # Store current number and index

    return []  # If no solution (though problem guarantees one)

# Test
nums = [2, 7, 11, 15]
target = 9
print(twoSum(nums, target))
```

```
[0, 1]
```

•[7]:
```python
#Longest Substring Without Repeating Characters
def length_of_longest_substring(s: str) -> int:http://localhost:8888/tree
    last_seen = {}
    left = 0
    max_len = 0

    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
        last_seen[ch] = right
        max_len = max(max_len, right - left + 1)

    return max_len

# Example
print(length_of_longest_substring("abcabcbb"))  # 3
print(length_of_longest_substring(""))          # 0
print(length_of_longest_substring("bbbbb"))     # 1
print(length_of_longest_substring("pwwkew"))    # 3
```

```
3
0
1
3
```

jupyter Untitled8 Last Checkpoint: 38 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

3

[8]:
```python
# Merge Intervals
def merge_intervals(intervals):
    if not intervals:
        return []

    # Step 1: Sort intervals by start time
    intervals.sort(key=lambda x: x[0])

    # Step 2: Initialize result with first interval
    merged = [intervals[0]]

    # Step 3: Iterate and merge
    for current in intervals[1:]:
        last = merged[-1]
        if current[0] <= last[1]:  # Overlap check
            last[1] = max(last[1], current[1])  # Merge ends
        else:
            merged.append(current)  # No overlap, add new

    return merged

# Test with example
intervals = [[1,3], [2,6], [8,10]]
print(merge_intervals(intervals))  # Output: [[1, 6], [8, 10]]
```

```
[[1, 6], [8, 10]]
```

jupyter Untitled8 Last Checkpoint: 39 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[10]:
```python
# Product of Array Except Self
def productExceptSelf(nums):
    n = len(nums)
    result = [1] * n

    # Step 1: Left products
    left = 1
    for i in range(n):
        result[i] = left
        left *= nums[i]

    # Step 2: Right products
    right = 1
    for i in range(n - 1, -1, -1):
        result[i] *= right
        right *= nums[i]

    return result


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    nums = list(map(int, input("Enter numbers (space-separated): ").split()))

    output = productExceptSelf(nums)
    print("Output:", output)
```

```
Enter numbers (space-separated):  1 2 3 4
Output: [24, 12, 8, 6]
```

jupyter Untitled8 Last Checkpoint: 39 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[10]:
```python
# Product of Array Except Self
def productExceptSelf(nums):
    n = len(nums)
    result = [1] * n

    # Step 1: Left products
    left = 1
    for i in range(n):
        result[i] = left
        left *= nums[i]

    # Step 2: Right products
    right = 1
    for i in range(n - 1, -1, -1):
        result[i] *= right
        right *= nums[i]

    return result


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    nums = list(map(int, input("Enter numbers (space-separated): ").split()))

    output = productExceptSelf(nums)
    print("Output:", output)
```

```
Enter numbers (space-separated):  1 2 3 4
Output: [24, 12, 8, 6]
```

jupyter Untitled8 Last Checkpoint: 39 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[11]:

```python
# Valid Parentheses Checker
def isValid(s):
    stack = []
    mapping = {
        ')': '(',
        '}': '{',
        ']': '['
    }

    for char in s:
        if char in mapping:
            # closing bracket
            if not stack or stack[-1] != mapping[char]:
                return False
            stack.pop()
        else:
            # opening bracket
            stack.append(char)

    return len(stack) == 0


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    s = input("Enter parentheses string: ")

    if isValid(s):
        print("Valid")
    else:
        print("Invalid")
```

Restart the kernel and run all cells

```
Enter parentheses string:  ({[]})
Valid
```

jupyter Untitled8 Last Checkpoint: 40 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
Enter parentheses string: ([]])
Valid
```

[12]:

```python
# Kth Largest Element
import heapq

def findKthLargest(nums, k):
    min_heap = []

    for num in nums:
        heapq.heappush(min_heap, num)

        if len(min_heap) > k:
            heapq.heappop(min_heap)

    return min_heap[0]


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    nums = list(map(int, input("Enter numbers (space-separated): ").split()))
    k = int(input("Enter k: "))

    result = findKthLargest(nums, k)
    print(f"{k}th largest element is:", result)
```

```
Enter numbers (space-separated):  2 4 7 8
Enter k:  2
2th largest element is: 7
```

jupyter Untitled8 Last Checkpoint: 40 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[13]:
```
7. # Search in Rotated Sorted Array
def search(nums, target):
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2

        if nums[mid] == target:
            return mid

        # Left half is sorted
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1

        # Right half is sorted
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    nums = list(map(int, input("Enter numbers (space-separated): ").split()))
    target = int(input("Enter target: "))

    index = search(nums, target)

    if index != -1:
        print("Target found at index:", index)
    else:
        print("Target not found")
```

```
Enter numbers (space-separated):  4 5 6 7 0 1 2
Enter target:  5
Target found at index: 1
```

jupyter Untitled8 Last Checkpoint: 41 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[14]:
```python
# Group words that are anagrams
def groupAnagrams(strs):
    anagram_map = {}

    for word in strs:
        key = ''.join(sorted(word))  # sorted string as key

        if key not in anagram_map:
            anagram_map[key] = []

        anagram_map[key].append(word)

    return list(anagram_map.values())


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    strs = input("Enter words (space-separated): ").split()

    result = groupAnagrams(strs)
    print("Grouped Anagrams:", result)
```

```
Enter words (space-separated):  eat tea tan ate nat bat
Grouped Anagrams: [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

jupyter Untitled8 Last Checkpoint: 41 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[15]:
```python
# Sliding Window Maximum (Hard)
from collections import deque

def maxSlidingWindow(nums, k):
    dq = deque()   # stores indices
    result = []

    for i in range(len(nums)):
        # Remove indices out of window
        if dq and dq[0] < i - k + 1:
            dq.popleft()

        # Remove smaller elements (not useful)
        while dq and nums[dq[-1]] < nums[i]:
            dq.pop()

        # Add current index
        dq.append(i)

        # Add to result once window is full
        if i >= k - 1:
            result.append(nums[dq[0]])

    return result


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    nums = list(map(int, input("Enter numbers (space-separated): ").split()))
    k = int(input("Enter window size k: "))

    output = maxSlidingWindow(nums, k)
    print("Output:", output)
```

```
Enter numbers (space-separated):  1 3 -1 -3 5 3 6 7
Enter window size k:  3
Output: [3, 3, 5, 5, 6, 7]
```

```
Enter window size k:  3
Output: [3, 3, 5, 5, 6, 7]
```

[16]:

```python
# Minimum Platform Problem
def minPlatforms(arr, dep):
    n = len(arr)

    arr.sort()
    dep.sort()

    i = 0   # arrival pointer
    j = 0   # departure pointer

    platforms = 0
    max_platforms = 0

    while i < n and j < n:
        # Train arrives
        if arr[i] <= dep[j]:
            platforms += 1
            max_platforms = max(max_platforms, platforms)
            i += 1
        else:
            # Train departs
            platforms -= 1
            j += 1

    return max_platforms


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    arr = list(map(int, input("Enter arrival times: ").split()))
    dep = list(map(int, input("Enter departure times: ").split()))

    result = minPlatforms(arr, dep)
    print("Minimum platforms needed:", result)
```

```
Enter arrival times:  900 940 950 1100 1500 1800
Enter departure times:  910 1200 1120 1130 1900 2000
Minimum platforms needed: 3
```

```python
# Detect Cycle in List
class ListNode:
    def __init__(self, val):
        self.val = val
        self.next = None


def hasCycle(head):
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next          # move 1 step
        fast = fast.next.next     # move 2 steps

        if slow == fast:
            return True

    return False


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    # Creating a linked list manually:
    # 1 -> 2 -> 3 -> 4 -> 5
    #           ↑       ↓
    #           ← ← ← ← ←   (cycle)

    head = ListNode(1)
    second = ListNode(2)
    third = ListNode(3)
    fourth = ListNode(4)
    fifth = ListNode(5)

    head.next = second
    second.next = third
    third.next = fourth
    fourth.next = fifth
    fifth.next = third   # 🔁 cycle created here

    if hasCycle(head):
        print("Cycle detected")
    else:
        print("No cycle")
```

```
Cycle detected
```

```
    else:
        print("No cycle")
```

```
Cycle detected
```

[18]:

```python
# Top K Frequent Elements
import heapq
from collections import Counter

def topKFrequent(nums, k):
    # Step 1: Count frequency
    freq = Counter(nums)

    # Step 2: Min heap (frequency, element)
    min_heap = []

    for num, count in freq.items():
        heapq.heappush(min_heap, (count, num))

        if len(min_heap) > k:
            heapq.heappop(min_heap)

    # Step 3: Extract elements
    result = [num for count, num in min_heap]
    return result


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    nums = list(map(int, input("Enter numbers (space-separated): ").split()))
    k = int(input("Enter k: "))

    output = topKFrequent(nums, k)
    print("Top K Frequent Elements:", output)
```

```
Enter numbers (space-separated):  1 1 1 2 2 3
Enter k:  2
Top K Frequent Elements: [2, 1]
```

jupyter Untitled8 Last Checkpoint: 43 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
Enter k:  2
Top K Frequent Elements: [2, 1]
```

[19]:

```python
#Word Break Problem (Dynamic Programming)
def wordBreak(s, wordDict):
    word_set = set(wordDict)   # for fast lookup
    dp = [False] * (len(s) + 1)
    dp[0] = True   # empty string is valid

    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break

    return dp[len(s)]


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    s = input("Enter string: ")
    wordDict = input("Enter dictionary words (space-separated): ").split()

    if wordBreak(s, wordDict):
        print("String can be segmented (Valid)")
    else:
        print("Cannot be segmented (Invalid)")
```

```
Enter string:  saptaparnichatterjee
Enter dictionary words (space-separated):  sapta parni chatter jee saptaparnichatterjee
String can be segmented (Valid)
```

jupyter Untitled8 Last Checkpoint: 43 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
String can be segmented (Valid)
```

[20]:

```python
# Longest Palindromic Substring
def longestPalindrome(s):
    if not s:
        return ""

    start, end = 0, 0

    def expand(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return right - left - 1   # length

    for i in range(len(s)):
        len1 = expand(i, i)       # odd length
        len2 = expand(i, i + 1)   # even length
        max_len = max(len1, len2)

        if max_len > (end - start):
            start = i - (max_len - 1) // 2
            end = i + max_len // 2

    return s[start:end + 1]


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    s = input("Enter string: ")

    result = longestPalindrome(s)
    print("Longest Palindromic Substring:", result)
```

```
Enter string:  babad
Longest Palindromic Substring: aba
```

jupyter Untitled8 Last Checkpoint: 43 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[21]:
```python
# Trapping Rain Water
def trap(height):
    left, right = 0, len(height) - 1
    left_max, right_max = 0, 0
    water = 0

    while left < right:
        if height[left] < height[right]:
            if height[left] >= left_max:
                left_max = height[left]
            else:
                water += left_max - height[left]
            left += 1
        else:
            if height[right] >= right_max:
                right_max = height[right]
            else:
                water += right_max - height[right]
            right -= 1

    return water


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    height = list(map(int, input("Enter heights: ").split()))

    result = trap(height)
    print("Trapped Water:", result)
```

```
Enter heights:  0 1 0 2 1 0 1 3 2 1 2 1
Trapped Water: 6
```